

Programming Languages: C/C++, Python, Java, Rust, Bash
Frameworks: Next.js/React, Node.js, Express.js, Ruby on rails, Angular.js
Technologies: Linux, Git, Docker, Jenkins, AWS (S3, ECS, RDS, SQS), Kubernetes, Apache Kafka, gRPC, Microservices architecture, Web Sockets, Unit Test, Embedded Systems, Cuda
Database: PostgreSQL, MongoDB, Redis
Certifications: PCEP, PCAP, AWS Solutions Architect, AWS Certified Developer

EDUCATION

University of California - Davis June 2025

- BS Computer Science

Relevant Coursework

- OOP in C++, Machine Learning, Computer Architecture, IoT Systems, Operating Systems, Data Structures & Algorithms, Discrete Math, Probability & Statistics, Linear Algebra, Multivariable Calculus, Database Systems, Assembly language, Network systems, Software engineering, Web development, Theory of Computation

WORK EXPERIENCE

Nvidia June 2025 - September 2025

Software Engineer Intern - Server Firmware Manageability Santa Clara, CA

- Engineered an SSH-based collector for **NVDebug** that transparently deploys diagnostic shell scripts to BMCs, executes them remotely, and retrieves multi-gigabyte log bundles—automating log acquisition across 100+ nodes and slashing manual triage effort by 75%
- Enabled Ansible Inventory file support for multiple internal tools including **NVDebug**, **NVSSVT**, and **RAS Tool**, allowing for log triage and collection on multi-DUT racks.
- Built **4factory** dashboards for the Nvidia-Observability stack, allowing for visualization and tracking of GB200 lab systems. Cut down on system debugging and fault identification by **90%** by preventing manual log search.
- Left-shifted validation and testing of several internal tools like **NVDebug**, **NVSSVT**, and **NVVCT** by executing these against **DCSIM** (Datacenter Simulation)

Aviz Networks June 2024 - September 2024

Software Engineer Intern - NWSLA team San Jose, CA

- Enhanced Network Service Level Agreement ecosystem for pushing calculated latencies across multiple endpoints in the network to the cloud in **pure C++** across a collaborative team of highly experienced senior developers and participated in **agile** practices
- Optimized NWSLA Collector to measure, aggregate, and retrieve latencies at the speed of **nanoseconds** using **gRPC** channels, giving rise to ultra-low latency networks
- Designed and optimized streaming pipeline to push latencies to **PostGres** using **Apache Kafka** saving **30%** on storage by leveraging **Avro** serialization
- Derived and automated unit tests using **Google Test** for NWSLA agents and collectors ensuring correct functionality
- Developed REST APIs using **pistache.io** enabling a seamless user interface and efficient communication with collectors

EcoCAR EV Challenge September 2022 - September 2023

Vehicle Software Engineer Davis, CA

- Wrote **Python/C++** scripts for data logging and CAN bus data extraction streamlining relevant data search
- Constructed **RTMaps** diagrams for sensor fusion based on Kalman filters, reducing false object detection by **15%**
- Conducted **BSM** simulation via V2X connectivity, identifying key bottlenecks in self-driving algorithms
- Developed 3D roadway/highway scenarios in **Roadrunner** (MATLAB) for sensor fusion testing covering versatile situations

PROJECTS

Navigation System | (C++) January 2024 - March 2024

- Developed a high-performance navigation system using Dijkstra’s algorithm with priority queue optimizations, achieving a **30%** reduction in route computation time for large-scale **OSM** datasets.
- Implemented **Test-Driven Development (TDD)** with **Google Test**, achieving **98%** code coverage and ensuring robustness across edge cases.
- Streamlined build and deployment using **CMake** and **Docker**, enabling scalable and reproducible containerized environments.
- Designed a modular architecture to support future enhancements, including **A algorithm integration*** and real-time traffic updates.

L-Store HTAP Database | (Python) September 2024 - December 2024

- Designed and implemented a high-performance Hybrid Transactional and Analytical Processing (**HTAP**) database in **Python** based on the L-Store architecture, optimizing for real-time **OLTP** and **OLAP** workloads
- Developed a **multi-version, lineage-based** storage system that enables contention-free updates and lazy staging, improving transactional consistency and query efficiency
- Achieved a **50%** increase in transaction throughput and reduced query execution time by **35%**, leveraging optimized indexing, concurrency control, and efficient page storage mechanisms.
- Optimized storage by implementing **append-only tail pages** and compression techniques, improving data retrieval speeds by **40%** and reducing storage costs by **30%**